University of Trento
Master's Degree in Computer Science

UNIVERSITÀ
DI TRENTO

Data Visualization
Final Report

Data Visualization Lab
A.Y. 2025/2026

Davide Donà	264467
Andrea Blushi	264468
Lorenzo Di Berardino	265984

August 2026

Contents

1 Introduction

While Citi Bike data captures the pulse of New York City's urban mobility, its true value is often buried.

This project bridges the gap by transforming raw data into an interactive web application that provides immediate, actionable insights. Powered by an efficient data pipeline, the application presents this information across four distinct views:

- **Map:** A multi-layer spatial view of geographic features.
- **Temporal:** A time-series tool designed to reveal usage habits and trends.
- **Weather:** An interface correlating weather conditions with ridership.
- **Footprint:** A sustainability dashboard evaluating environmental impact.

To support data exploration, all views can be dynamically filtered by date range, user type, and bike type.

2 Data Pipeline

Aggregating and visualizing a decade of Citi Bike trip data introduces several challenges:

- **Scale:** Hundreds of millions of trip records must be filtered and aggregated fast enough to keep the dashboard interactive.
- **Format Shifts:** Multiple data sources follow different packaging and column conventions, which have to be reconciled into a single schema.
- **Data Quality:** Each source carries missing values, outliers, and coverage gaps that would otherwise surface as misleading visuals.

Our pipeline addresses these challenges, enabling us to fulfill the core objectives outlined in our project proposal.

2.1 Pipeline Overview

The pipeline turns raw sources into dashboard-ready data through three sequential stages, illustrated in Figure 1:

1. **Data Acquisition** gathers the static datasets evaluated in our technical report: the trip archives, hourly weather, bike-route geometries, and a station metadata snapshot. Each download is cached on disk with a freshness window, so repeated runs re-fetch only what has aged.
2. **Cleaning and Normalization** leverages *Polars* to unify heterogeneous source schemas, filter invalid records, and enrich each trip with key filter dimensions: date, hour, and day of the week. The pipeline then exports the cleaned data as compressed, monthly-partitioned *Parquet* files.
3. **Precomputation** aggregates cleaned trips into compact hourly and monthly summaries stored in *PostgreSQL*. These include hourly ridership trends, station-level turnover, and station-pair trajectories, alongside synchronized hourly weather metrics to contextualize usage. Because monthly releases must be

merged into these summaries without corrupting past runs, we write every table via dimension-keyed upserts, ensuring idempotent reloads and safe pipeline retries.

In line with our project proposal, a *GitHub Action* executes this complete sequence monthly and **publishes** the updated *Docker* image to a public repository, keeping the dashboard continuously up to date.

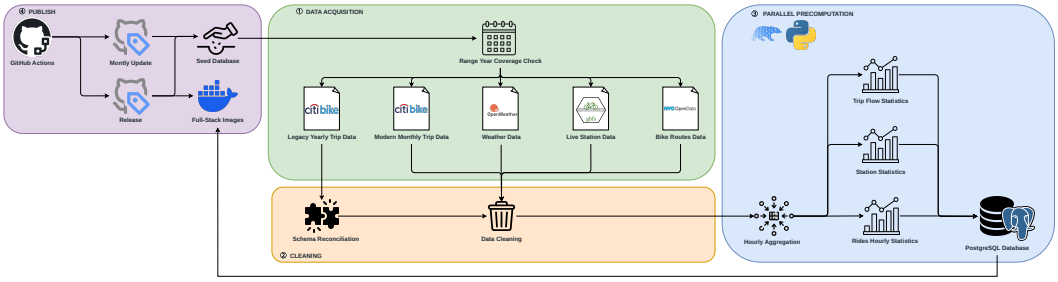

Figure 1: Data pipeline: the end-to-end architecture.

2.2 Data Acquisition

Of the four datasets gathered in this stage, the trip archives dominate: a decade of rides spans hundreds of millions of records, while the hourly weather, bike-route, and station metadata sources each arrive as a single, comparatively small file or API response. To keep this asymmetry from exhausting standard hardware memory resources, the pipeline processes the trip archives through the following steps:

1. **Streamed Downloading:** Downloads each archive to a temporary file in small chunks.
2. **Monthly Processing:** Converts the data to a *Parquet* file one month at a time.
3. **Immediate Cleanup:** Deletes each intermediate file after loading.

To accelerate this process, downloads overlap with database ingestion while worker threads compute aggregates in parallel.

To handle monthly releases without reprocessing the entire archive, the system tracks existing coverage and automatically expands requested ranges. This guarantees a contiguous timeline while keeping each update proportional to the new data alone.

2.3 Cleaning and Normalization

Raw trip records inherit the data quality issues identified in our technical report, from missing fields and format drift to statistical outliers.

Missing Values Any records missing station, timestamp, user, or bike type data are immediately discarded. Because the affected volume is negligible, discarding these rows preserves data integrity.

Legacy Schema Reformatting Beyond missing fields, format itself drifted over the years: Citi Bike archives arrive in two packaging formats, yearly bundles containing monthly archives (legacy), and flat monthly files (modern). The pipeline automatically handles both structures, emitting a single data frame per month.

We reconcile these schema variations by retaining only the intersection of columns, discarding any fields not present across all years.

Trip Outliers Once records share a common schema, outliers remain the last distortion to filter out. Three filters prevent extreme or invalid records from distorting duration and distance metrics:

- **Negative Durations:** Records with timestamps where the end precedes the start are discarded.
- **Round Trips:** Rides sharing the same origin and destination are removed, as their zero straight-line distance would artificially affect distance averages.
- **Upper-Tail Extremes:** Long-duration and long-distance outliers are removed using a 95% z-score threshold, calculated per month for each metric. Using a hard threshold would have been too aggressive, as the distribution could vary significantly across months and years.

2.4 Precomputation

Turning cleaned trip rows into dashboard-ready aggregates meant first establishing a complete time base, then keeping every query fast.

Time and Weather Spine To accurately calculate hourly utilization rates, the total number of elapsed hours within a given timeframe must be established. Our initial design derived time buckets directly from the trip logs, operating on the assumption that at least one ride occurred per hour. However, hours with zero activity, frequently corresponding with adverse weather conditions, were omitted, obscuring quiet periods. To resolve this, we generated an independent, continuous hourly calendar spine and joined the trip data to it.

Similar to the time spine, visualizing ridership against weather conditions necessitates a two-dimensional grid. Each hour of the spine is labelled with its observed weather condition, and hours without rides contribute zero-valued rows. This yields a complete, rectangular dataset that renders reliably without requiring complex visualization workarounds.

Precomputing Statistics With a complete hourly base in place, the remaining challenge was speed. Our initial design relied on *Polars* to directly query raw *Parquet* files without a database layer. However, every user interaction triggered expensive full-table scans across hundreds of millions of raw rows, leading to unacceptable latency.

We shifted the heavy computational load to the ingestion pipeline by precomputing data at a one-hour granularity. Because this granularity matches the finest detail required by the dashboard, we eliminate the need for on-the-fly full-table scans.

Not every query requires fine detail: a citywide yearly total needs only monthly sums, whereas a full hour-by-day-of-week breakdown is rarely used. Scanning the finest hourly table for these coarser requests wastes compute proportional to the gap between required and available resolutions. To keep query costs proportional to the resolution needed, three additional variants are rolled up alongside the hourly table: by month, hour of day, and day of week. Each variant is aggregated from the same cleaned trips to ensure consistency across all four. Then, the backend routes each request to the smallest table able to answer it (Section 3).

3 Backend

The backend exposes the pipeline's precomputed tables through a *FastAPI* service. The intermediate *Parquet* files are deleted post-ingestion, leaving *PostgreSQL* as the single source of truth.

Live station availability is the sole exception; it is fetched from the feed on each request behind a short-lived cache rather than being read from the database.

3.1 Live and Historical Data Reconciliation

The system reconciles discrepancies between the GBFS live feed and Citi Bike historical trip archives through three rules:

- **Identifier Alignment:** All records are mapped to shared station identifiers (`short_name`), resolving mismatches between the two sources.
- **Operational Filtering:** Live and spatial views display only stations actively reporting as installed, renting, and returning. All other stations are dropped from these results, even if they appear in historical archives. However, unmatched stations are retained for citywide temporal and aggregate statistics that do not require map locations.
- **Capacity Derivation:** We calculate station capacity dynamically using live dock counters to keep our availability ratios accurate. This avoids reliance on static metadata, which can be missing or outdated, leaving capacity undefined for some stations.

3.2 Storage Engine

Serving all of this data, live and historical alike, still comes down to a single engine choice for the database itself. While an embedded engine like *DuckDB* could query raw files without a precomputation layer, *PostgreSQL* proved to be the better fit for our serving layer:

- **Concurrency and Scalability:** *PostgreSQL* natively manages concurrent user requests, whereas embedded engines are single-process and risk bottlenecks under load.
- **Point Query Performance:** Combined with our precomputed aggregates, the remaining query workloads consist of small lookups where indexed *PostgreSQL* table access outperforms columnar file scans.
- **Architectural Fit:** A persistent, shared database aligns better with our deployment architecture than running an embedded query engine.
- **Engine Efficiency:** Because *Polars* already powers our ingestion pipeline, introducing *DuckDB* as a second analytical engine was unnecessary.

A related decision concerns bike lane shapes: as geographic data, they would normally call for a spatial database extension such as *PostGIS*. Because we never query these shapes spatially, we store each one as plain text and parse it into coordinates in the backend before serving, keeping the storage free of heavy dependencies.

Query Routing Beyond the choice of engine, its precomputed tables are not all the same size. As described in Section 2, the pipeline rolls up several redundant variants of each statistic at different time resolutions. The backend routes each

request to the smallest variant able to answer it: a citywide yearly total, for instance, resolves straight from the monthly rollup rather than scanning the full hourly table. In this way, we trade a modest amount of extra storage for reduced scan volumes at query time.

4 Visualization

The following visualizations each address a distinct question about the Citi Bike system. This section presents them in turn, detailing the specific encodings, supported interactions, and underlying design decisions involved. Before covering each view individually, the following two subsections introduce the shared technical implementation and consistent design language behind them.

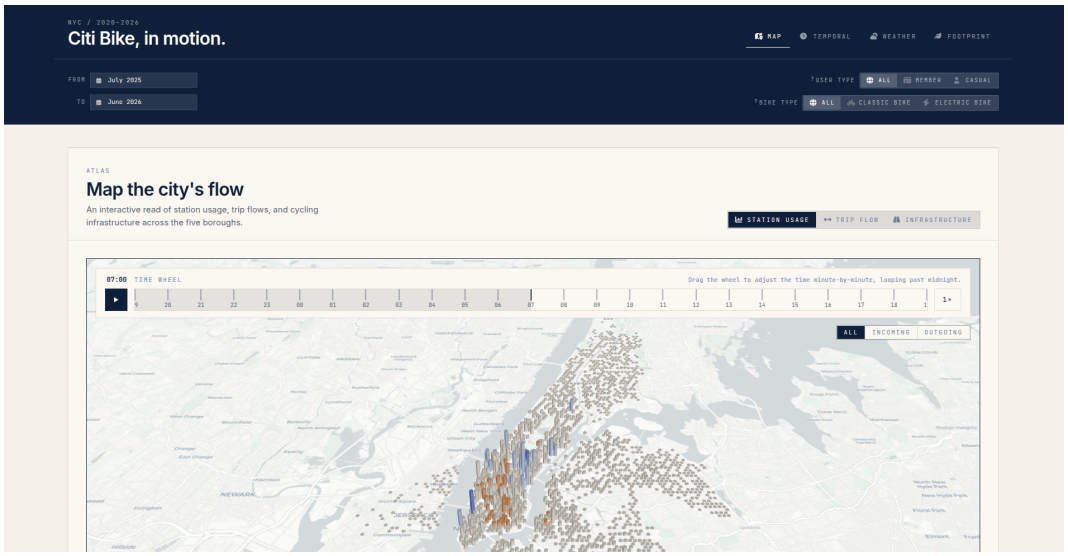

Figure 2: Dashboard: the starting view.

4.1 Implementation

The frontend is a *React* application built with *Vite* and routed via *React Router*. Layout and visual design tokens are managed through a unified *Tailwind CSS* module, ensuring consistent styling across interface components, charts, and maps.

Each view employs a library tailored to its specific data structures:

- **deck.gl** renders *WebGL* map layers, maintaining interactive performance across thousands of stations and arcs.
- **Plotly** generates three-dimensional surfaces and distribution plots.
- **Chart.js** and custom canvas to draw simple 2D charts.

Although the original proposal specified *D3.js* for non-map charts, using *Plotly* and *Chart.js* for standard visuals saved development time. *D3.js* provides no ready-made charts, so every axis, scale, and update animation has to be written by hand. Both libraries animate data changes on their own, and a single duration and easing setting keeps motion consistent across every chart.

4.2 User Experience

To maintain consistency, all components share a standardized layout skeleton: a descriptive header, a primary display, secondary plots, and a collapsible reading guide. Notably, these secondary plots expand upon the initial proposal by providing additional context around the core insights.

Visual Design To prioritize data clarity over decorative clutter, the interface uses a clean design:

- **Color Usage:** Neutral tones for the layout, reserving bright colors strictly to highlight inside charts.
- **Typography:** Clear, readable fonts applied consistently across all views.
- **Animation:** Minimal motion to enhance the user experience without adding distraction.

Interaction Model Because the dashboard is exploratory, ensuring immediate clarity without enforcing a narrative was a primary design concern. We adopted a three-part interaction model:

- **Guide:** Each view has a collapsible "How To Read It" panel (Figure 3) with three practical hints, giving users a clear starting point for exploration.
- **Tooltips:** Embedded hints and hover-triggered tooltips surface exact values and fine-grained details on demand, permitting to have a complete overview without overwhelming.
- **Defaults:** Sensible default states offer immediate global insights, while interactive filters, toggles, and linked selections enable deeper inspection on demand.

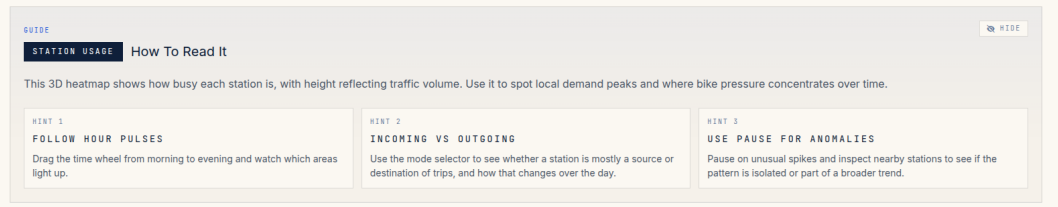

Figure 3: Dashboard: the reading guide.

4.3 Header

The global header incorporates two primary filter controls (Figure 4):

- **Date Range Picker:** A monthly calendar selector with a 12-month maximum range. Maximum ranges too short would limit trend visibility, while ranges too long would slow down performance.
- **User and Bike Type Selectors:** A pair of selectors filtering data by membership status (member or casual) and bike type (classic, electric, or both).

Changing any global filter automatically reloads the charts while keeping selections active across pages. The precomputation described in Section 2 keeps these reloads fast.

Response Caching Beyond how fast each query resolves, performance also depends on how often the same request repeats. Our dashboard requires multiple queries across multiple pages and filters, which can lead to repeated requests for the same data. The frontend caches (*TanStack Query*) responses keyed by their exact filter set, allowing revisited views to render instantly.

Cross-filtering These global filters narrowed the cross-filtering planned in our proposal. To address this, we added view-specific filters that operate independently of the header. Additionally, filters that were only relevant to specific views were removed from the global header and relocated.

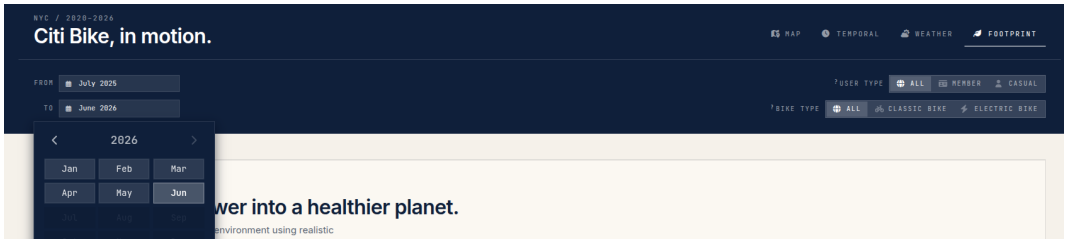

Figure 4: Dashboard: the global header.

4.4 Map

As the dashboard's default view (Figure 2), the spatial map offers an immediate, impactful geographic overview. Multiple relevant spatial insights can be extracted from the Citi Bike dataset, including station usage, trip flow, and infrastructure. We chose to break down the analysis into three focused map layers rather than combining them into one view.

4.4.1 Station Usage

This layer establishes where demand concentrates across the network, summarizing station activity into a form that remains clear at city scale.

Hexagon Heatmap By aggregating individual stations into hourly bins, we visualize the overall activity distribution across space and time of day. Figure 5 presents a 3D heatmap of elevated hexagons, where height encodes absolute usage and color highlights deviation from mean activity. This dual encoding makes it easy to spot spatial anomalies in both absolute and relative terms.

Other visualizations were considered, including choropleth maps and 2D heatmaps. However, displaying thousands of stations and millions of rides often led to overcrowded, unreadable views. For instance, a 2D heatmap resulted in heavy overlap, turning the whole city red without any specific zoom. In contrast, our 3D encoding overcomes this issue by using multiple visual channels.

Map Controls While effective for overall spatial trends, the 3D hexagon heatmap alone abstracts away time and other key filters. To restore this depth, the layer uses precomputed hourly trip counts (Section 2) and includes a mode selector to isolate incoming, outgoing, or total trips. This reveals both heavy-traffic stations and top destinations.

Temporal navigation is managed through an interactive time wheel with play controls to animate daily average usage. Because the hourly raw data created an inconsistent transition, the time wheel linearly interpolates between adjacent hourly averages. This design trade-off provides a continuous, fluid animation at the expense of showing synthesized intermediate values rather than raw measurements.

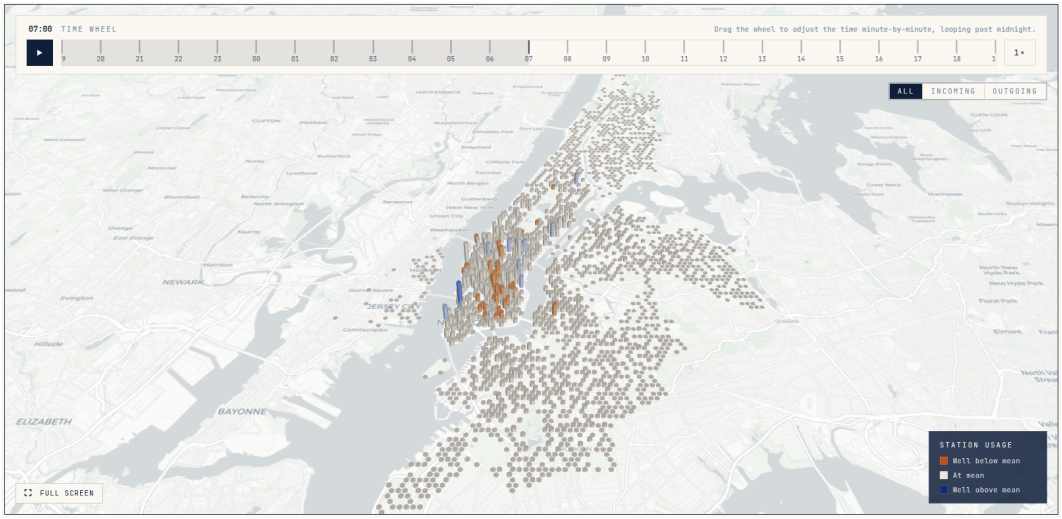

Figure 5: Station usage: the hexagon heatmap.

Peak Hour and Station Rankings To complement the spatial view, two auxiliary plots provide targeted insights (Figure 6):

- **Station Usage by Peak Hour:** A bar chart showing how many stations reach peak activity during each hour of the day. Because selecting a single hour on the 3D map masks city-wide patterns, this view offers an immediate, high-level summary of peak activity distribution across the network.
- **Busiest Stations:** A ranked bar chart highlighting the top stations by average daily trip volume. This eliminates 3D spatial occlusion, allowing users to instantly identify top-performing locations.

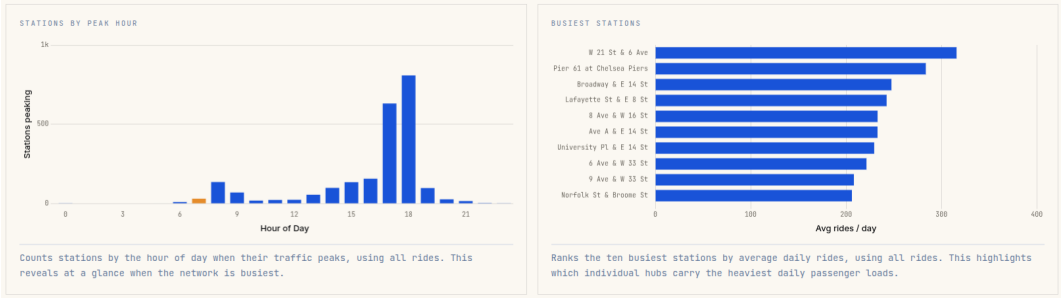

Figure 6: Station usage: the supplemental plots.

4.4.2 Trip Flow

While the previous layer quantifies activity at individual stations, this one examines the qualitative connections between them.

Corridor Arcs This layer renders origin-destination arcs representing trips between stations, normalized to daily averages for easy comparison across different

time periods (Section 2). Arc width, together with hue, encodes trip volume, dynamically scaled against the busiest corridor in the current view. Arc height and curvature carry no quantitative data, as they serve exclusively to disentangle overlapping paths and maintain visual clarity across dense networks.

Citywide Overview The initial implementation rendered the entire network simultaneously, producing extreme visual clutter that obscured individual flows. To resolve this, the second prototype hid all connections by default, revealing top corridors only upon station selection. While this restored clarity, it completely erased the broader overview and discarded secondary routes.

The final design reconciles both perspectives by rebalancing visual encoding rather than truncating underlying data. Initial loading presents a citywide overview of the primary transit corridors (Figure 7), providing an immediate macroscopic view of high-volume routes prior to user interaction.

Station Focus Selecting an individual station isolates its connected corridors and automatically re-frames the camera view around it (Figure 8). Upon selection, color semantics shift from absolute volume to directional flow asymmetry, categorizing links as predominantly inbound, outbound, or balanced. To avoid misinterpreting minor fluctuations, routes are classified as directional only when traffic strongly favors one side, and as balanced otherwise.

In doing this, we encountered a major data limitation. Since database rows store station pairs in a canonical order, raw data would randomly assign inbound or outbound labels. We resolve this by dynamically reorienting data relative to the selected station.

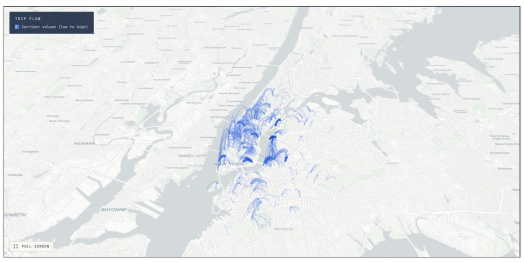

Figure 7: Trip flow: the citywide overview.

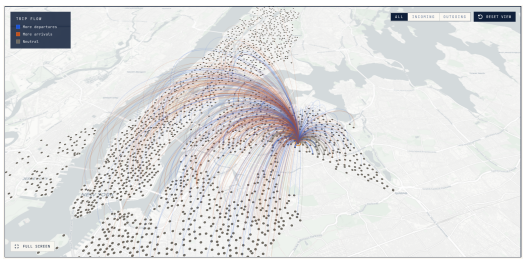

Figure 8: Trip flow: the station focus.

Summary Cards and Corridor Ranking Two secondary analytical panels complement the spatial view (Figure 9):

- **Summary Cards:** Display high-level aggregates for the active context, including total daily rides, active corridor counts, flow-weighted median trip distance, and net directional shares for selected stations. This permits users to quickly assess the overall network state without inspecting individual arcs.
- **Ranked Corridor List:** A bar chart illustrating the top twelve corridors of the current selection. Interactive hovering highlights the corresponding spatial arc, while clicking a bar isolates that specific corridor by hiding all remaining network links. This provides a clear, focused view of the busiest routes.

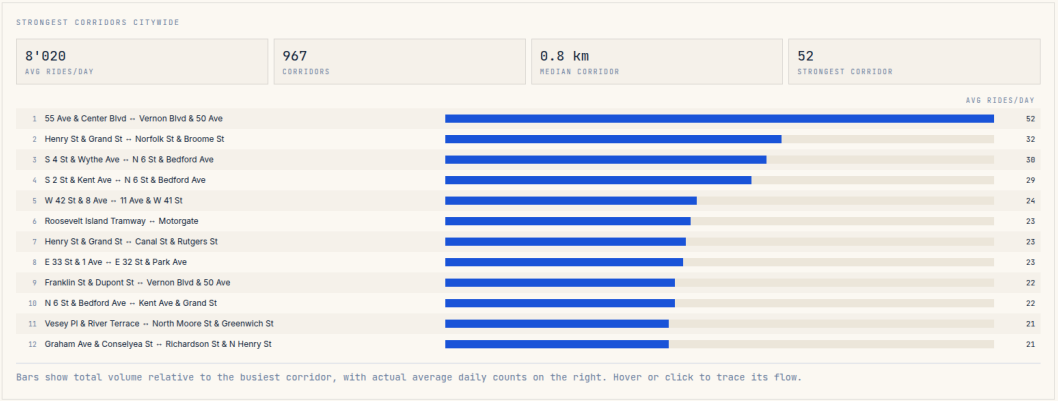

Figure 9: Trip flow: the supplemental plots.

4.4.3 Infrastructure

Although bike-lane data was excluded from our initial proposal, early analysis showed its value. Combining lane data with station and route network gave a full picture of the physical infrastructure. Additionally, the NYC DOT dataset provided complete, high-quality data, including exact installation and retirement dates for every segment.

Station Status and Bike Routes The default view displays real-time station capacity (Figure 10), highlighting nearly empty or full locations so users can spot supply risks at first glance. Then, by selecting an interactive toggle, users can overlay NYC bike routes (Figure 11). To prevent visual clutter, we hid the bike routes by default, since segments otherwise would have obscured station locations and their operational status, making them difficult to identify.

Station Sidebar Initially, station details were displayed only in hover tooltips. However, this approach quickly became cluttered and restricted the amount of information we could show.

To provide a comprehensive overview of each station, we introduced a contextual sidebar that opens upon selection, combining real-time and historical metrics (Figure 12):

- **Live Availability:** Reports effective capacity, open docks, and rentable bikes (disaggregated by classic and electric), alongside out-of-service counts.
- **Historical Profile:** Displays hourly and daily ride distributions, identifying peak usage hours, busiest days, and overall arrival-departure balance.
- **Top Connected Stations:** Ranks the station's five strongest corridors, in order to provide a quick overview of the station's connectivity. Selecting a corridor re-centers the interface on the partner station, allowing sequential exploration across connected routes.

Year Slider A temporal slider completes the infrastructure layer, allowing users to filter bike lanes by installation year. By rendering only segments active in a selected year, the layer transforms from a static overview into a historical view of NYC's evolving cycling network.

However, this feature introduced a major challenge. Segments dated `01/01/1900` predate formal records, so removing them would delete long-standing paths. To avoid misleading interpretations, we kept these segments and added a note explaining that this early date represents a historical baseline, not an active construction year.

Moreover, integrating real-time and historical datasets introduced an unavoidable temporal mismatch. While bike routes include installation and retirement timestamps, station metadata is available only as a live snapshot (Section 3). Consequently, the temporal slider filters the route network exclusively while holding station locations at their current state, avoiding speculative reconstruction of past station positions.

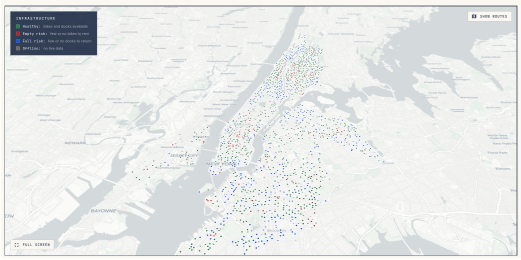

Figure 10: Infrastructure: the station status.

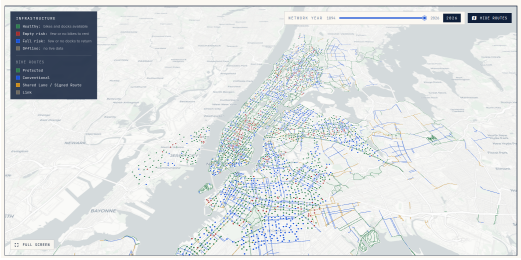

Figure 11: Infrastructure: the bike routes overlay.

Figure 12: Infrastructure: the station sidebar.

Network Composition Charts Three supplementary plots extend the bike lane infrastructure view (Figure 13):

- **Network Changes by Year:** A diverging bar chart counting segments installed above the axis and retired below it, giving the temporal overview the map alone cannot convey.

- **Segments by Borough:** A ranked breakdown comparing network size across the five boroughs.
- **Segments by Facility Class:** The same count split by protection level, showing how much of the network is physically separated.

Figure 13: Infrastructure: the supplemental plots.

4.5 Temporal

Setting geography aside, this view asks when the system is used.

Surface Graph The temporal view features a 3D surface plot mapping activity across days of the week and hours of the day. Users can switch the metric between average trip volume, duration, distance, and speed. Furthermore, the plot can be freely rotated to inspect trends from any angle, and hovering over any point reveals exact values (Figure 14). This visualization was considered the main view, as its peaks instantly reveal major activity trends at a glance.

Comparison Mode Although global filters isolate specific user or bike types, constantly switching between them imposes a heavy cognitive load, as users must memorize the previous view to compare it with the current one. To address this, we expanded our initial proposal by adding a multi-surface comparison view. Overlaying surfaces in the same space (Figure 15) allows direct comparison. To prevent overlapping layers from obscuring each other, each surface uses a distinct color scheme with semi-transparent top layers.

To prevent ambiguities, comparison mode enforces three key structural safeguards:

- **Duplicate Prevention:** Prohibits adding duplicate layers, as identical surfaces would only occlude each other without adding analytical value.
- **Filter Consistency:** Clears all pinned comparison layers whenever global header filters change, ensuring stale data is not compared against a newly selected population.

- **Canvas Persistence:** Prevents the removal or hiding of the final active surface so the visual canvas is never left empty.

Figure 14: Temporal: the day-hour surface.

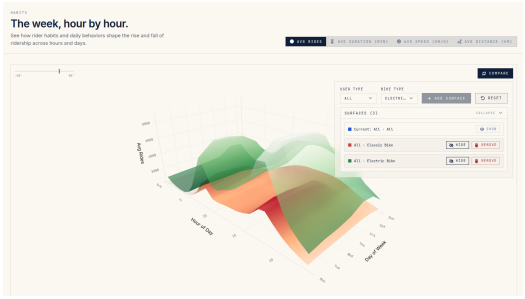

Figure 15: Temporal: the comparison mode.

Histograms The 3D surface plot shows overall shape well, but it makes the detailed distribution of the data hard to see. To address this, two marginal histograms aggregate data by weekday or hour (Figure 16).

To make the relationship between the surface and histograms clear, all three views are interactively linked. Hovering over a surface cell highlights its corresponding histogram bars, while clicking a bar pins that day or hour. Pinned selections project their distribution onto the opposite histogram and overlay a trace line across the 3D surface. In comparison mode, the histogram bars match the colors of each active surface (Figure 17).

Line Chart Extreme weather events, like heavy storms, highlighted a gap in our initial design. Because aggregated charts smooth out this kind of outlier, single-day peaks from weather or holidays were easily missed. To fix this, we introduced a dedicated line chart plotting every date individually. This allows us to instantly spot daily outliers, cross-reference data with real-world events, and perform accurate root-cause analysis without losing broader context.

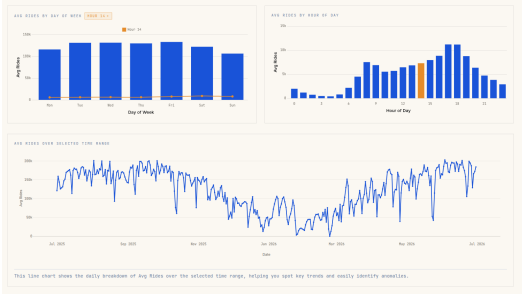

Figure 16: Temporal: the supplemental plots.

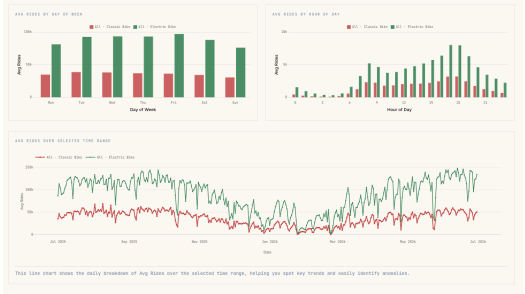

Figure 17: Temporal: the supplemental plots in comparison mode.

4.6 Weather

This view isolates how much weather changes ridership.

Condition Scatter The primary visualization is a scatter plot where each point represents a WMO weather code. Selecting appropriate axes for the scatter plot required several iterations to make anomalies both visible and explainable. Initial

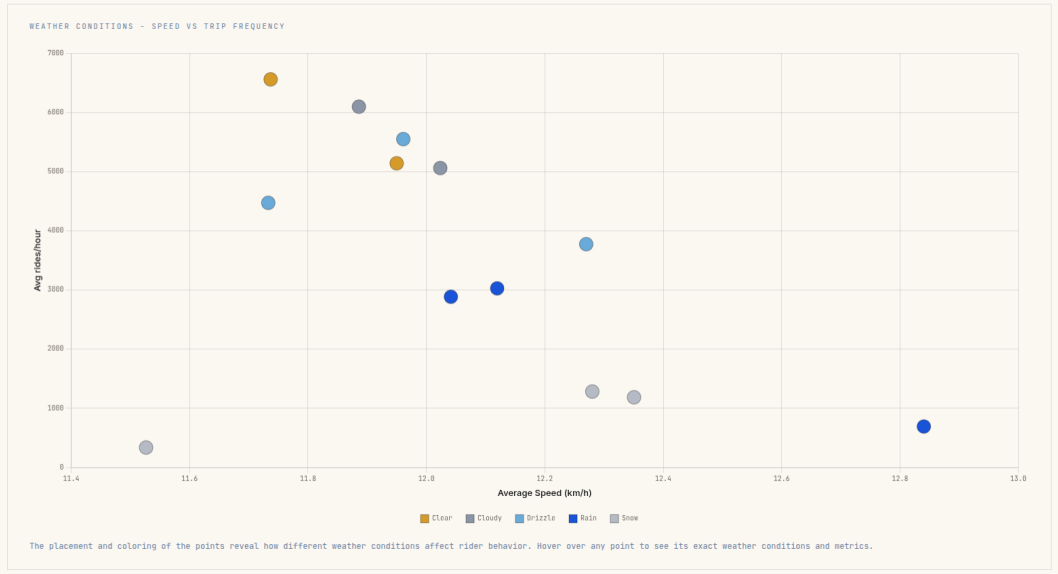

Figure 18: Weather: the condition scatter.

attempts (such as plotting total rides against temperature) were dominated by common conditions like clear skies, collapsing rare events near the origin. On the other hand, average duration and average distance increased together in a clear, straight line, rather than forming distinct groups. Normalizing volume to average hourly rides and pairing it with average trip speed resolved these issues.

Each dot is color-coded by weather condition family (Figure 18), grouping them into high-level categories (e.g., rain, snow, thunderstorm) to simplify interpretation. This permits intuitive color semantics rather than raw numerical codes, allowing conditions with similar behavioral impacts to form distinct visual clusters.

Then, interactive controls enable deeper data exploration. Clicking a legend category isolates a specific condition family, while hovering over any data point surfaces a detailed tooltip with weather conditions and trip statistics. This provides a compact summary of how weather suppresses or sustains global cycling activity.

Ridgeline Distributions Although our proposal made the ridgeline chart the primary view, we found it hard to interpret at first glance. On the other hand, while the scatter plot immediately highlighted key trends, it could not show how usage changed over time. Rather than removing the ridgeline chart, we moved it to a secondary view to complement the scatter plot.

To capture those missing time shifts, the ridgeline chart plots usage curves for each weather type (Figure 19), complete with an interactive toggle to switch between hourly and weekly views. During development, we decided to scale each ridge to its own peak. This design choice made it easy to compare activity patterns between rare events, such as thunderstorms, and baseline clear days. However, this introduced a key trade-off, as scaling hid absolute ride volumes. To prevent readers from mistaking relative patterns for total demand, we added clear chart annotations.

Temperature and Rain Response Standard weather categories often group too many days together, hiding crucial details about absolute volume and how temperature or rain specifically affect rider behavior. To uncover these finer details, two supplementary plots examine continuous weather variables directly (Figure 20):

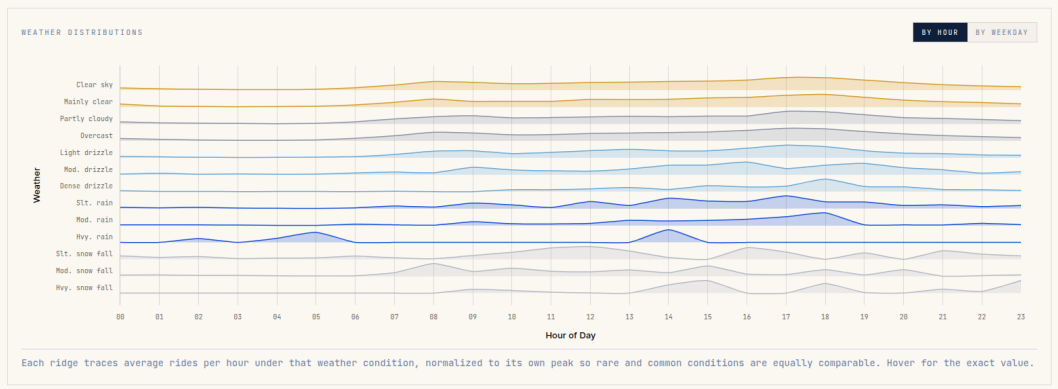

Figure 19: Weather: the ridgeline distributions.

- **Temperature Response:** A line chart showing average hourly rides for every one-degree temperature change.
- **Rain Impact:** A bar chart showing the percentage drop in ridership across different rain levels compared to dry weather.

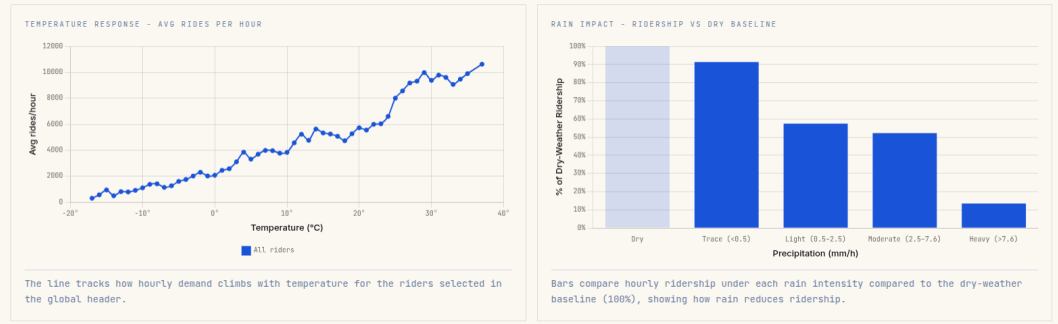

Figure 20: Weather: the supplemental plots.

4.7 Footprint

The footprint view translates ridden distance into environmental impact using published emission factors. Because real-world car substitution rates vary significantly across studies and rider demographics, fixing this rate to a single static value would be misleading. To reflect this real-world uncertainty, an interactive slider allows readers to adjust the substitution rate between 5% and 25%, dynamically updating all visualizations to show a range of plausible outcomes.

Tiles and Emission Charts Three headline tiles summarize total distance ridden, avoided CO_2, and replaced car trips (Figure 21). A horizontal bar chart compares how much emission that same distance produces across transport modes, holding distance constant to highlight efficiency differences. Finally, a cumulative chart tracks daily avoided CO_2 over time, with a shaded band showing the range between the minimum and maximum substitution rates and a dotted line tracing the user's selected setting.

We chose this dual-layer design to prevent readers from mistaking rough estimates for exact facts. Showing the full range alongside a moving line lets users explore different assumptions while keeping real-world uncertainty clearly in view.

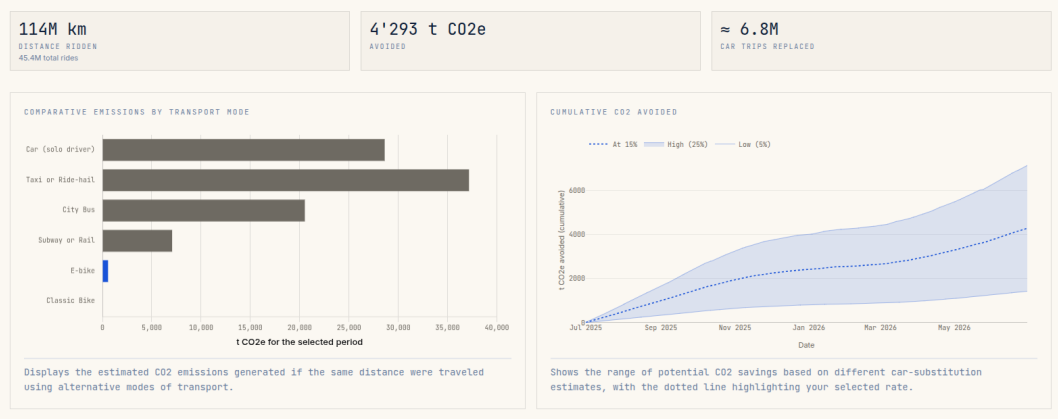

Figure 21: Footprint: the emission tiles and charts.

Pictogram Because raw carbon tonnage is hard to picture, a pictogram translates these numbers into relatable real-world benchmarks: urban trees absorbing carbon, a person's annual emissions, and powered LED bulbs (Figure 22).

To keep the display clean, each row targets roughly 24 icons by rounding unit values to clean steps (like 1, 2, or 5 times a power of ten). Adjusting the slider simply changes the number of icons shown. However, changing the date range rescales what each icon represents to keep the row length manageable.

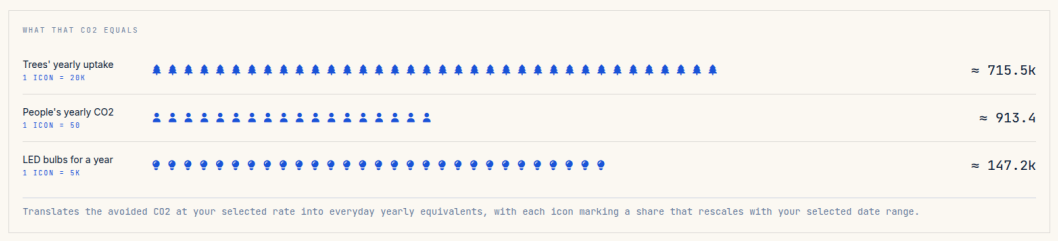

Figure 22: Footprint: the pictogram.

Assumptions Box Unlike other views, this page displays modeled estimates rather than directly measured values. To prevent users from mistaking avoided CO_2 figures for direct measurements, we prioritized explicit transparency. An assumptions box outlines all baseline inputs, including transport mode emission factors, substitution rates from cited literature, and computational exclusions (Figure 23). Inline citations provide direct links for verification.

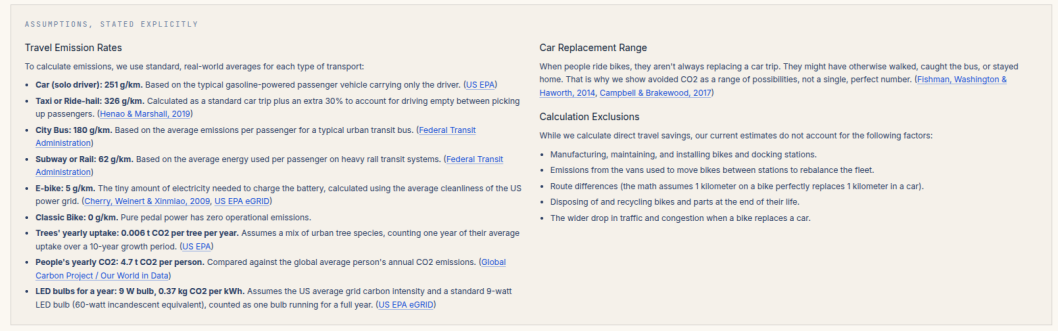

Figure 23: Footprint: the assumptions box.

5 Conclusion

Beyond the technical decisions covered so far, delivering this project also depended on how effectively we worked as a team. This section reflects on that collaboration, the opportunities we would pursue next, and how well the final result met our original goals.

5.1 Collaboration

Building a full-stack project required organized teamwork through *GitHub*. We used issues to track features, separate branches to work independently, and pull requests to review code before merging. This helped us catch bugs early and share responsibility across the team.

Because almost every visualization changed multiple times, we focused on keeping our code maintainable. We wrote modular code, documented key interfaces, and continuously cleaned up our codebase.

Finally, we automated our workflow using *GitHub Actions*. On every pull request, the pipeline runs backend tests against a temporary *PostgreSQL* database, checks the frontend code, and builds our LaTeX documentation.

5.2 Future Work

Throughout the project, we identified several opportunities that would enhance the dashboard's usefulness and analytical depth.

Actual Trip Routes Because the dataset records only origin, destination, and duration, exact travel paths are unknown. GPS traces or routing engines would map trips onto the street network, transforming straight corridors into realistic cycling routes and enriching spatial visualizations.

Station Availability History As noted in Section 3, dock occupancy is limited to live snapshots. Persisting this data over time would reveal availability patterns, showing when shortages occur, how long they last, and how effectively rebalancing responds.

Demand Forecasting The current dashboard is entirely descriptive. However, as our temporal and weather views demonstrate, ridership responds predictably to weather, time, and day. Adding a forecasting model to these features would predict station-level demand, moving the tool from descriptive analysis to operational planning.

5.3 Final Considerations

The project met its main goal. The interface highlights key patterns immediately, including weekday commutes, rain sensitivity, and differences between members and casual riders. Throughout the project, we chose clear, practical solutions over complex ones, creating a system that is easy to understand, maintain, and expand.